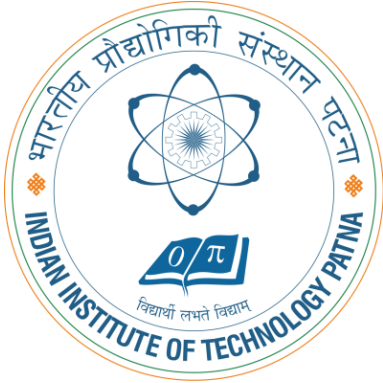


CS5102: FOUNDATIONS OF COMPUTER SYSTEMS

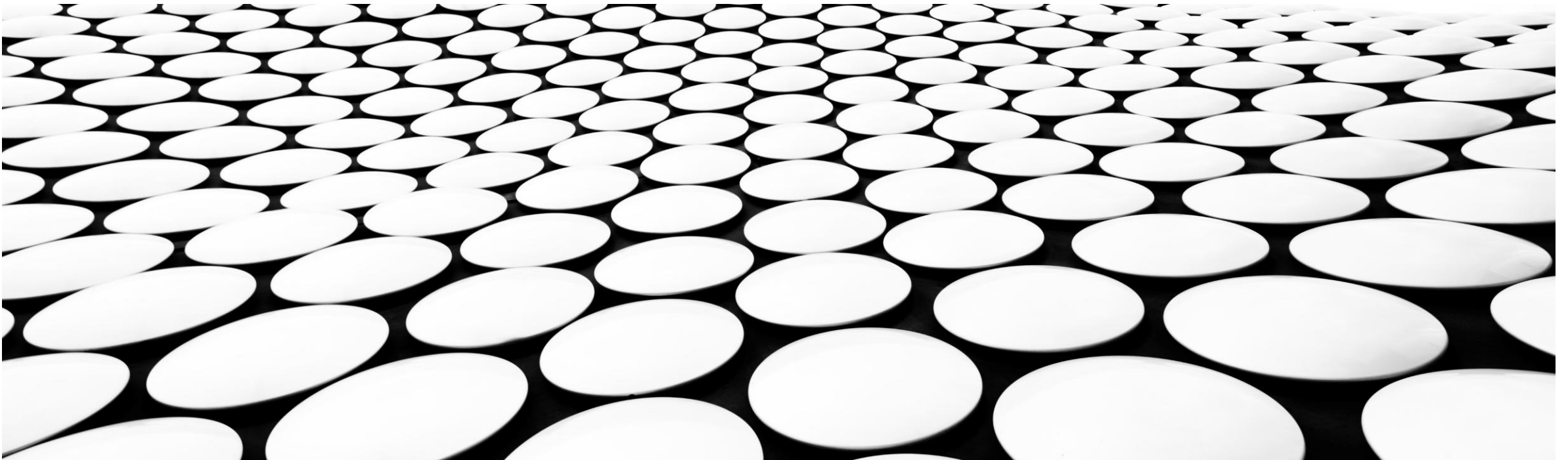


TOPIC-8: PROGRAMMING-III

DR. ARIJIT ROY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

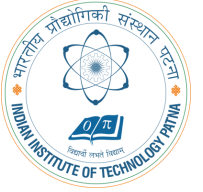
INDIAN INSTITUTE OF TECHNOLOGY PATNA





RECAP

IF STATEMENT



High-level code

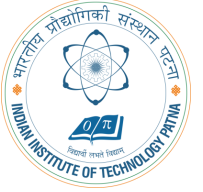
```
if (i == j)
    f = g + h;

f = f - i;
```

MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h
# $s3 = i, $s4 = j
```

IF STATEMENT



High-level code

```
if (i == j)
    f = g + h;

f = f - i;
```

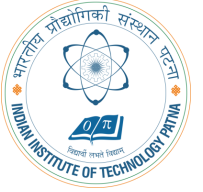
MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h
# $s3 = i, $s4 = j
bne $s3, $s4, L1
add $s0, $s1, $s2

L1: sub $s0, $s0, $s3
```

Notice that the assembly tests for the opposite case ($i \neq j$) than the test in the high-level code ($i == j$).

IF/ELSE STATEMENT



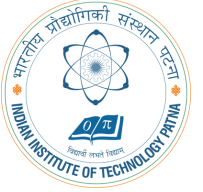
High-level code

```
if (i == j)
    f = g + h;
else
    f = f - i;
```

MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h
# $s3 = i, $s4 = j
```

IF/ELSE STATEMENT



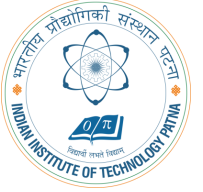
High-level code

```
if (i == j)
    f = g + h;
else
    f = f - i;
```

MIPS assembly code

```
# $s0 = f, $s1 = g, $s2 = h
# $s3 = i, $s4 = j
        bne $s3, $s4, L1
        add $s0, $s1, $s2
        j   done
L1:     sub $s0, $s0, $s3
done:
```

WHILE LOOPS



High-level code

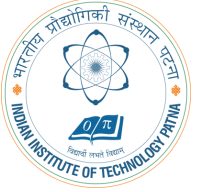
```
// determines the power
// of x such that 2x = 128
int pow = 1;
int x = 0;

while (pow != 128) {
    pow = pow * 2;
    x = x + 1;
}
```

MIPS assembly code

```
# $s0 = pow, $s1 = x
```

WHILE LOOPS



High-level code

```
// determines the power
// of x such that 2x = 128
int pow = 1;
int x    = 0;

while (pow != 128) {
    pow = pow * 2;
    x = x + 1;
}
```

MIPS assembly code

```
# $s0 = pow, $s1 = x

        addi $s0, $0, 1
        add  $s1, $0, $0
        addi $t0, $0, 128
while:   beq  $s0, $t0, done
        sll  $s0, $s0, 1
        addi $s1, $s1, 1
        j    while
done:
```

Notice that the assembly tests for the opposite case (`pow == 128`) than the test in the high-level code (`pow != 128`).

FOR LOOPS

The general form of a for loop is:



FOR LOOPS

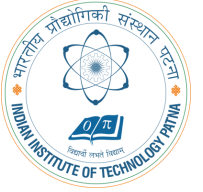


The general form of a for loop is:

```
for (initialization; condition; loop operation)    loop body
```

- `initialization`: executes before the loop begins
- `condition`: is tested at the beginning of each iteration
- `loop operation`: executes at the end of each iteration
- `loop body`: executes each time the condition is met

FOR LOOPS



High-level code

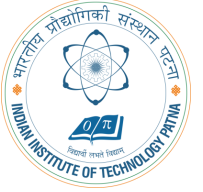
```
// add the numbers from 0 to 9
int sum = 0;
int i;

for (i=0; i!=10; i = i+1) {
    sum = sum + i;
}
```

MIPS assembly code

```
# $s0 = i, $s1 = sum
```

FOR LOOPS



High-level code

```
// add the numbers from 0 to 9
int sum = 0;
int i;

for (i=0; i!=10; i = i+1) {
    sum = sum + i;
}
```

MIPS assembly code

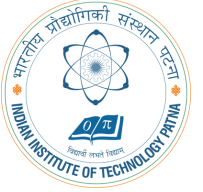
```
# $s0 = i, $s1 = sum
        addi $s1, $0, 0
        add  $s0, $0, $0
        addi $t0, $0, 10
for:     beq  $s0, $t0, done
        add  $s1, $s1, $s0
        addi $s0, $s0, 1
        j    for
done:
```

Notice that the assembly tests for the opposite case ($i == 10$) than the test in the high-level code ($i \neq 10$).



PROGRAMMING PART-III

LESS THAN COMPARISONS



High-level code

```
// add the powers of 2 from 1
// to 100  int sum = 0;  int
i;

for (i=1; i < 101; i = i*2) {
    sum = sum + i;
}
```

set on less than

```
slt $1,$2,$3
if ($2<$3) $1=1;
else $1=0
```

Test if less than.

If true, set \$1 to 1.

Otherwise, set \$1 to 0.

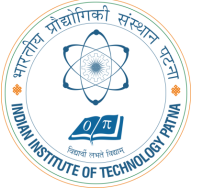
MIPS assembly code

```
# $s0 = i, $s1 = sum
addi $s1, $0, 0
addi $s0, $0, 1
addi $t0, $0, 101
loop:  slt $t1, $s0, $t0
       beq $t1, $0, done
       add $s1, $s1, $s0
       sll $s0, $s0, 1
       j  loop
done:
```

$\$t1 = 1$ if $i < 101$.

ARRAYS

- Useful for accessing large amounts of similar data
- Array element: accessed by *index*
- Array *size*: number of elements in the array



ARRAYS



- 5-element array
- **Base address** = 0x12348000 (address of the first array element, `array[0]`)
- First step in accessing an array: load base address into a register

0x12340010	array[4]
0x1234800C	array[3]
0x12348008	array[2]
0x12348004	array[1]
0x12348000	array[0]

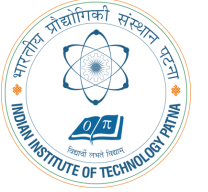
ARRAYS

// high-level code

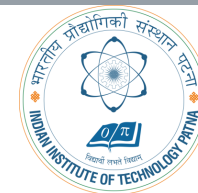
```
int array[5];  
array[0] = array[0] * 2;  
array[1] = array[1] * 2;
```

MIPS assembly code

```
# array base address = $s0
```



ARRAYS



// high-level code

```
int array[5];  
array[0] = array[0] * 2;  
array[1] = array[1] * 2;
```

MIPS assembly code

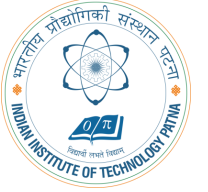
array base address = \$s0

```
lui  $s0, 0x1234          # put 0x1234 in upper half of $s0  
ori  $s0, $s0, 0x8000     # put 0x8000 in lower half of $s0
```

```
lw   $t1, 0($s0)          # $t1 = array[0]  
sll  $t1, $t1, 1          # $t1 = $t1 * 2  
sw   $t1, 0($s0)          # array[0] = $t1
```

```
lw   $t1, 4($s0)          # $t1 = array[1]  
sll  $t1, $t1, 1          # $t1 = $t1 * 2  
sw   $t1, 4($s0)          # array[1] = $t1
```

ARRAYS USING FOR LOOPS



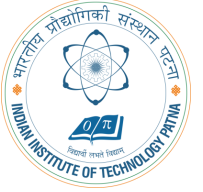
```
// high-level code
```

```
int array[1000];  int i;
```

```
for (i=0; i < 1000; i = i + 1)
```

```
array[i] = array[i] * 8;
```

ARRAYS USING FOR LOOPS



// high-level code

```
int array[1000]; int i;
```

```
for (i=0; i < 1000; i = i + 1)
array[i] = array[i] * 8;
```

set on less than

```
slt $1,$2,$3
```

```
if ($2<$3) $t=1;
```

```
else $t=0
```

Test if less than.

If true, set \$t to 1.

Otherwise, set \$t to 0.

MIPS assembly code

```
# $s0 = array base address, $s1 = i
```

```
# initialization code
```

```
lui $s0, 0x23B8 # $s0 = 0x23B80000
```

```
ori $s0, $s0, 0xF000 # $s0 = 0x23B8F000
```

```
addi $s1, $0, 0 # i = 0
```

```
addi $t2, $0, 1000 # $t2 = 1000
```

```
loop:
```

```
slt $t0, $s1, $t2 # i < 1000?
```

```
beq $t0, $0, done # if not then done
```

```
sll $t0, $s1, 2 # $t0 = i * 4 (byte offset)
```

```
add $t0, $t0, $s0 # address of array[i]
```

```
lw $t1, 0($t0) # $t1 = array[i]
```

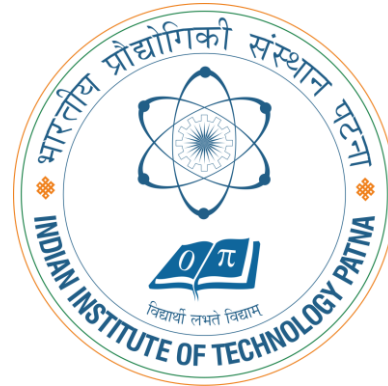
```
sll $t1, $t1, 3 # $t1 = array[i] * 8
```

```
sw $t1, 0($t0) # array[i] = array[i] * 8
```

```
addi $s1, $s1, 1 # i = i + 1
```

```
j loop # repeat
```

```
done:
```



THANK YOU!